

Week 3 Report — Data Quality Validation & Graceful Degradation

Arnav Mahale · AIPI 561 Operationalizing AI · Summer 2026

1. Issues found in the corrupted window ($\geq 2026-01-16$)

Baseline = the 6.08 M rows before `time_bucket = 2026-01-16`. Corrupted candidate window = 250 853 rows on/after that cutoff. All four issues are detected by `week3/validation/check_data_quality.py`.

#	Type	Severity	Evidence	Why it breaks the model
1	Duplicate rows	HIGH	8 134 fully duplicated rows + 10 085 (<code>zone</code> , <code>time_bucket</code>) key duplicates (0 in baseline)	Inflates aggregations; the demand profile would count the same 15-min bucket twice, biasing hourly averages and lag features.
2	Out-of-range <code>trip_count</code>	CRITICAL	353 negative values (impossible) and 311 values $> 5\,000$ (max 99 999 vs baseline max 310)	Negatives propagate into <code>lag_*</code> / <code>roll_mean_*</code> features for future predictions; the 99 999 outliers blow up mean-based features by 4 $\times$ and skew the model toward extreme forecasts.
3	<code>cbd_pricing_active</code> lost variance	HIGH	Was a 0/1 toggle (baseline mean 0.338, std 0.473); in the candidate window it's stuck at 1 for all 250 853 rows (std = 0)	The model can't learn from a constant — the feature contributes zero discriminative signal and any coefficient learned on it during week-2 training is now mis-applied to every prediction.
4	<code>is_holiday</code> rate spike	MEDIUM	Baseline holiday rate $\approx 3.9\%$ of rows; corrupted window $\approx 15.5\%$ ($\approx 4 \times$ baseline; only $\approx 4.5\%$ expected from	Pushes predictions into the “holiday” demand profile too often, suppressing weekday commuter demand estimates.

#	Type	Severity	Evidence	Why it breaks the model
			MLK + Presidents Day)	

2. Validation strategy

The validator (`DataQualityValidator`) layers six checks: schema, null-rate, hard value ranges, full-row + key duplicates, variance collapse on categorical features, and rate drift vs. baseline. Each detected issue carries a severity (critical / high / medium / low); `is_valid` is `False` only when at least one critical or high issue is present, so medium-severity drift does not block deployment.

Where validation runs (three layers, each catching a different failure mode):

1. **CI/CD** — `.github/workflows/validate-data.yml` pulls the latest upstream parquet from `gs://ops-ai-arnavmahale-data` and runs the validator + pytest suite. Critical/high issues fail the workflow and block any merge.
2. **API startup** — `week3/backend/data.py` calls `check_and_log_data_quality()` at module import; every issue is logged at WARNING/ERROR severity so operators see them in pod logs without the API ever blocking traffic.
3. **Inline cleanup at load** — `_clean_upstream()` runs immediately after `pd.read_parquet`, dedupes by `(PULocationID, time_bucket)` and clips `trip_count` to `[0, 500]`. Every cleaning action emits a WARNING with the row count affected.

3. Validation schedule — hourly cron

`cron: '0 * * * *'` runs the validation workflow once an hour, plus on every push that touches `week3/`, plus a `workflow_dispatch` for ad-hoc runs.

Justification: new upstream taxi data arrives roughly every 15 minutes, so running validation every 15 minutes would mostly re-check the same rows for 2–3 cycles before any new data appears — wasted compute. Daily is the other extreme: it could let a full day's worth of corruption flow into predictions before anyone notices. Hourly hits the sweet spot of catching corruption within ~60 minutes of arrival while staying well inside GitHub Actions free tier (~24 runs × ~2 minutes = ~48 free-tier minutes/day on a 2 000 min/month budget). Push-triggered runs catch validation logic changes before they ship.

4. Graceful degradation strategy

The API **never** crashes on bad data. Specifically:

- **Duplicates** → silently drop, keep first occurrence, log the count.
- **Out-of-range trip_count** → clip to `[0, 500]`, log the count below and above.
- **Variance collapse / rate drift** → log only; no fallback at the data layer because the feature is consumed downstream by the LightGBM model, which was trained on a feature that had variance. Logging surfaces the problem to the operator (and the CI block prevents redeploying a new model trained on this corrupted feature).
- **Parquet missing or unreadable** → `_load()` catches the exception, logs an error, and returns an empty profile so the FastAPI app still starts and `/health` returns 200. Endpoints serving from the empty profile return empty results rather than 500s.
- **LightGBM model missing** → `_load_model()` already returns `None`; forecast endpoints degrade to profile-based estimates only.

The principle is the one from this week's READING.md: graceful degradation is **transparent, not silent**. Every fallback writes to `logging` with the affected row count, so operators have actionable signal (the *what*, *how many*, and *severity*) even though the service never goes down.

5. What this design intentionally doesn't do

Honest limits to flag in the next iteration:

- The hourly workflow doesn't open a Slack/PagerDuty alert — failed CI is the only signal. Real prod would pipe the validator's structured `issues[]` output into the alerting backend.
- No correlation/feature-stability check (Layer 3 from the reading): a feature could stay structurally valid but stop predicting. Adding a Pearson-r vs baseline check on `lag_1h / roll_mean_1h` against `trip_count` is the natural next step.
- The runtime degradation handles *known* issue classes only. Novel corruption would still be served until the validator is updated.